Android Input Sensor

开发指南

版本号: 3.0

发布日期: 2025.09.18

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.11.11	AW	初始版本文档
1.1	2020.12.31	AW	更新部分配置路径
2.0	2023.03.16	AW1696	1. 增加 sensorhal 介绍。2. 完成 sensor 适配过程。3. 增加调试及常见问题。
2.1	2025.01.20	AW2078	title 增加 Input 字段。
3.0	2025.09.18	AW2467	1. 增加支持 Sensor 校准。2. 增加支持抬起唤醒。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1

2 驱动模块适配 **2**

3 Sensorhal 介绍 **3**
3.1 框架介绍 4
3.2 源码介绍 5
3.2.1 Google SensorHal AIDL 5
3.2.2 AW Sensorhal 5
3.2.3 sensors.h 介绍 6
3.2.3.1 SENSOR_STRING_TYPE 6
3.2.3.2 sensor_t 6
3.3 Sensor 类型 7
3.4 SensorInfo 7
3.5 SensorHal 工作流程 8
3.5.1 初始化 8
3.5.1.1 Sensor 驱动加载 9
3.5.1.2 自动匹配设备 10
3.5.1.3 创建 sensor 10
3.5.2 使能 Sensor 11
3.5.3 设置 delay 12
3.5.4 处理 event 12

4 支持 Sensorhal **14**
4.1 编译 14
4.2 添加 feature 14

5 适配 Sensor **16**
5.1 已实现 Sensor 16
5.1.1 加载驱动 16
5.1.2 添加 Sensor 信息 16
5.1.3 数据处理 16
5.1.4 AccelSensor 方向适配 17
5.1.4.1 方向配置文件 17
5.1.4.2 方向配置 18
5.1.4.3 方向适配 18

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2 未定义的 sensor 类型 20

6 支持 Sensor 校准 21

6.1 Sensor 校准原理 21
6.2 支持的传感器类型 21
6.2.1 加速度传感器，重力传感器，磁力传感器 21
6.2.2 光线传感器，距离传感器，压力传感器，温度传感器 21
6.3 Sensorhal 为一个新类型的 sensor 支持校准 22
6.4 校准功能配置开关 23
6.5 Android 校准功能支持 24
6.5.1 有哪些接口 24
6.5.1.1 setCalibrationData 24
6.5.1.2 getCalibrationData 24
6.5.1.3 clearCalibrationTmpData 24
6.5.1.4 clearCalibrationSstData 25
6.5.2 写入数据格式要求 25
6.5.2.1 加速度传感器，重力传感器，磁力传感器 25
6.5.2.2 光线传感器 25
6.5.2.3 温度传感器 25
6.5.2.4 压力传感器 26
6.5.2.5 距离传感器 26
6.5.3 APP 适配流程 26
6.5.3.1 权限声明和参数配置 26
6.5.3.2 如何获取实例化对象 26
6.5.3.3 使用 demo 示例 27
6.5.4 如何 Debug 28
6.5.4.1 确定数据是否写入 28
6.5.4.2 dumpsys sensorservice 的使用 29

7 抬起唤醒开发 31

7.1 Gsensor 抬起唤醒原理 31
7.2 功能所需硬件要求 31
7.3 MSA321 Gsensor 抬起唤醒 31
7.3.1 设备树配置 32
7.3.2 驱动适配 32
7.3.2.1 中断申请 32
7.3.2.2 注册新设备 33
7.3.2.3 设备节点创建 34
7.3.2.4 中断处理 35
7.3.2.5 休眠唤醒处理 38
7.3.3 效果调优 39
7.3.3.1 中断触发阈值调整 39
7.3.3.2 唤醒角度配置 40

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7.3.3.3 检测时长配置 40
7.4 Android 抬起唤醒功能支持 40
7.4.1 方案配置开关 40
7.4.1.1 菜单显示 41
7.4.1.2 功能默认值 41
7.4.2 Powerhal 配置文件节点 42
7.4.3 使用说明 43

8 调试方法 45
8.1 dumpsys sensorservice 45
8.2 remonut 后推库 46

9 常见问题 47
9.1 dumpsys sensorservice 能看到设备信息，app 却无法正确获得 sensor 47
9.2 节点无法写入，提示没有权限 47

广州星翼电子科技有限公司zdyzlinux

插　图

图 3-1　Sensor 框架 4
图 3-2　Sensor 框架 9
图 4-1　sensor_feature 15
图 6-1　sensorhal_logcat 打印 29
图 6-2　最近 sensor 事件 30
图 7-1　Gsensor 抬起唤醒流程图 31
图 7-2　raise_to_wake 44
图 9-1　节点路径 47
图 9-2　uevent 48

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 编写目的

为达到能快速使用的目的。文档主要对以下内容做了详细的讲解。

1.Input Sensor 的使用方法步骤，如何在 SensorHal 添加一个新的模组。

2. 不同类型的传感器（加速度、温度、重力、磁力、光线、压力、距离等）设计的统一的校准框架，如何进行接口调用。

3. 如何使用 Gsensor 来实现抬起唤醒功能，客户可参考对应章节中的适配步骤，完成 Gsensor 抬起唤醒的方案开发。

1.2 适用范围

本文档主要适用于全志 Android 15 及 16 以上平台。

抬起唤醒：

技巧

除 Gsensor 外，开发人员亦可基于文档描述步骤，开发基于陀螺仪或其他 Sensor 的抬起唤醒功能。

注意

Android15 上，部分功能需要打印补丁后支持，如有需求请联系全志 FAE 获取。

1.3 相关人员

相关的开发与维护人员应该仔细阅读本文档。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 驱动模块适配

驱动模块的适配，参考《Android Input 开发指南》；本文档主要介绍 Sensorhal 的开发适配。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 Sensorhal 介绍

SensorHal 是 google 原生支持的 Hardware 之一，主要用于管理 Linux Sensor，获取 event 并传递到 Framework，以供应用层使用，例如实现自动旋转，光距感应等。

在 AW 发布的 SDK 中，Android 13 以上 Sensorhal 使用 AIDL，在以往的版本使用 HIDL，本文主要基于 AIDL 进行介绍，对于与 HIDL 不一致且需要开发注意的不同点，会特别注明。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.1 框架介绍

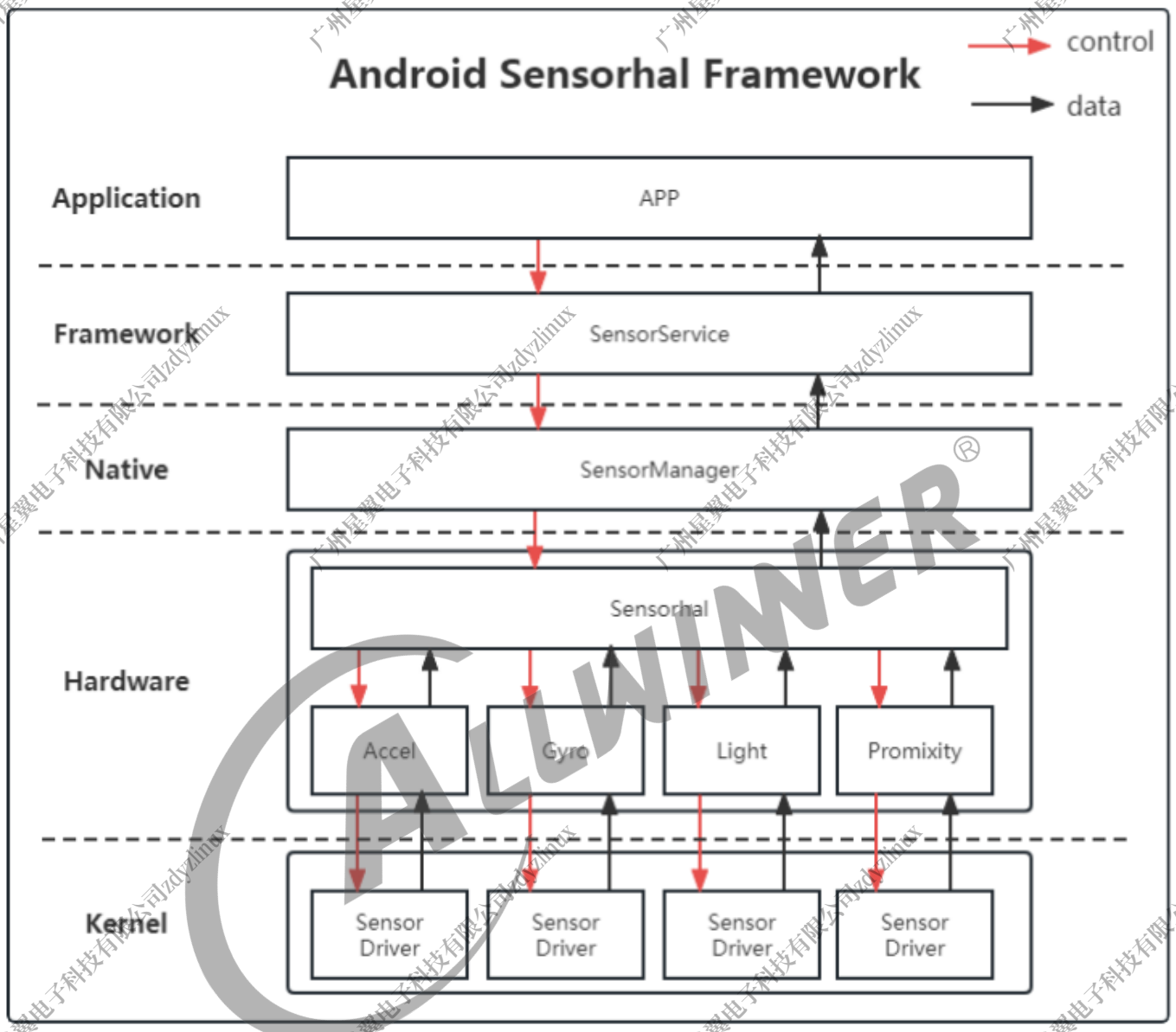

图 3-1: Sensor 框架

上图是 Android 中 Sensor 的控制-数据流程图，可看到 Sensorhal 的位置及作用，是处于一个承上启下的作用。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2 源码介绍

3.2.1 Google SensorHal AIDL

Google 原生代码路径，通常有 default 实现，支持的 Sensor 类型，Sensor 数据头文件等。

```
/hardware/interfaces/sensors/
```

aidl 源文件路径。

```
hardware/interfaces/sensors/aidl/android/hardware/sensors
```

其文件描述信息如下。

```
└── sensors
    ├── AdditionalInfo.aidl
    ├── DynamicSensorInfo.aidl
    ├── Event.aidl                    //event事件定义
    ├── ISensors.aidl                 //SensorHal的主文件，定义了Sensorhal的使用接口
    ├── ISensorsCallback.aidl
    ├── SensorInfo.aidl               //SensorInfo数据介绍
    ├── SensorStatus.aidl             //Sensor状态描述
    └── SensorType.aidl               //Sensor类型介绍，定义了当前支持的所有Sensor类型
```

3.2.2 AW Sensorhal

全志当前 Sensorhal 源码路径，主要是 Sensorhal 的实现。

```
hardware/aw/sensors/aw_sensors
```

其源码实现。

```
aidl/
├── impl                          //Sensorhal实现
│   ├── AccelSensor.cpp           //重力感应gsensor实现类
│   ├── AmbientTempSensor.cpp
│   ├── Android.bp
│   ├── include                   //头文件
│   ├── InputEventReader.cpp      //用于读取Event事件
│   ├── LightSensor.cpp
│   ├── MagnetometerSensor.cpp
│   ├── PressureSensor.cpp
│   ├── ProximitySensor.cpp
│   ├── Sensor.cpp                //Sensor的父类，定义了一个Sensor的标准接口
│   ├── SensorInit.cpp            //Sensor初始化，保存了预定义Sensor信息，具有加载Sensor驱动，初始化
    Sensor信息功能
│   └── Sensors.cpp               //Sensorhal的主要接口，用于管理所有的Sensor
└── service                       //service实现
    ├── Android.bp
    ├── main.cpp
    ├── sensors-default-aw.rc     //sensor启动rc类
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
└── sensors-default-aw.xml      //兼容性矩阵
```

3.2.3 sensors.h 介绍

senosrs.h 是 google 原生的 sensor 库文件，主要是定义了许多 sensor 相关的信息，路径如下。

```
/hardware/libhardware/include/hardware/sensors.h
```

这里介绍常用的一些信息。

3.2.3.1 SENSOR_STRING_TYPE

定义了 sensor 的 string 名称，用于 SensorInfo 上报。

```
#define SENSOR_STRING_TYPE_ACCELEROMETER                "android.sensor.accelerometer"
#define SENSOR_STRING_TYPE_MAGNETIC_FIELD               "android.sensor.magnetic_field"
#define SENSOR_STRING_TYPE_ORIENTATION                  "android.sensor.orientation"
#define SENSOR_STRING_TYPE_GYROSCOPE                    "android.sensor.gyroscope"
#define SENSOR_STRING_TYPE_LIGHT                        "android.sensor.light"
#define SENSOR_STRING_TYPE_PRESSURE                     "android.sensor.pressure"
#define SENSOR_STRING_TYPE_TEMPERATURE                  "android.sensor.temperature"
#define SENSOR_STRING_TYPE_PROXIMITY                    "android.sensor.proximity"
```

3.2.3.2 sensor_t

该结构体主要用于保存 sensor 信息。

```
struct sensor_t {
    const char*     name;                   //sensor名字
    const char*     vendor;                 //产品，一般描述是哪家的
    int             version;                //版本号
    int             handle;                 //用于索引
    int             type;                   //sensor类型
    float           maxRange;               //最大范围
    float           resolution;             //分辨率
    float           power;                  //通常为0，当前未使用
    int32_t         minDelay;               //最小采样间隔时间，单位为ns
    uint32_t        fifoReservedEventCount;
    uint32_t        fifoMaxEventCount;
    const char*    stringType;              //sensor类型的string，参考<SENSOR_STRING_TYPE>章节
    const char*    requiredPermission;      //需要的权限，默认为0

    #ifdef __LP64__
       int64_t maxDelay;                    //最大采样间隔时间，单位为ns
    #else
       int32_t maxDelay;
    #endif

    #ifdef __LP64__
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        uint64_t flags;                     //flag，通常有wakeup,contious,on-change等
    #else
        uint32_t flags;
    #endif

    void*           reserved[2];
};
```

3.3 Sensor 类型

当前 Android 版本支持的主要常用 Sensor 类型如下。

```
ACCELEROMETER;              //重力感应gsensor
MAGNETIC_FIELD;             //磁力感应
GYROSCOPE;                  //陀螺仪
LIGHT;                      //光感
PROXIMITY;                  //距离感应
```

支持的类型 type 描述信息具体可查阅如下文件。

```
hardware/interfaces/sensors/aidl/android/hardware/sensors/SensorType.aidl
```

说明

霍尔传感器不属于常规的 Sensor，霍尔传感器适配请参考《Android Input 开发指南》。

3.4 SensorInfo

在 Sensorhal 中，需要预先定义使用的 Sensor 信息用于初始化，当前所有可能使用的 Sensor 信息存放位置如下。

```
hardware/aw/sensors/aw_sensors/aidl/impl/SensorInit.cpp
```

Sensor 信息是以结构体数组存放的，参考定义如下。

```
struct SensorData AccelSensorConfigList[] = {
}
```

其中，SensorData 的定义如下。

```
struct SensorData {
    char sensorName[64];                //Sensor的名字，表现为dev/input/下的名字
    char classPath[128];                //Sensor的sys/class/input/inputX路径，这里均为空，运行时获取
    char enableNodeName[64];            //Sensor的enable节点名字，使能时与classPath拼接使用
    char delayNodeName[64];             //Sensor的delay节点名字，使能时与classPath拼接使用
    float priData;                      //Sensor的私有数据，用于sensor的数组转换，如gsensor的转换数据等
    struct sensor_t sensorInfo;         //Sensor信息
};
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

在 SensorData 中还存在 sensor_t 结构体，可参考章节。

如下是一个 AccelSensor 举例。

```
    {
        // sc7a20
        "sc7a20",                                   //dev/input下的驱动名称
        "",                                         //sys/class/input下的路径，默认空，自动生成
        DEFAULT_ENABLE,                             //enable节点，默认为"enable"
        DEFAULT_DELAY,                              //delay节点，默认为"delay"
        32.0f,                                      //pri数据，用于转换上报的事件
        {                                           //sensor_t结构体
            "lis7a20 detect Accelerometer",         //产品名称
            "SL",                                   //产品供应商，这里是SL
            1,                                      //版本为1
            0,                                      //handle，填0，自动生成
            SENSOR_TYPE_ACCELEROMETER,              //sensor类型
            (GRAVITY_EARTH * 4.0f),                 //最大范围为4倍重力
            GRAVITY_EARTH / 1024.0f,                //最小分辨率
            0.2f,                                   //power，当前未使用
            10000,                                  //最小采样间隔，单位ns，这里则是10ms
            0,                                      //fifoReservedEventCount,为0即可
            0,                                      //fifoMaxEventCount,为0即可
            SENSOR_STRING_TYPE_ACCELEROMETER,       //参考<SENSOR_STRING_TYPE>
            0,                                      //权限，为0即可
            1000000,                                //最大采样间隔，这里是1s
            SENSOR_FLAG_CONTINUOUS_MODE,            //flags，这里为continous
            {},                                     //reserved，空即可
        },
    },  // sc7a20
```

3.5 SensorHal 工作流程

3.5.1 初始化

初始化工作流程如图所示。

广州星翼电子科技有限公司zdyzlinux

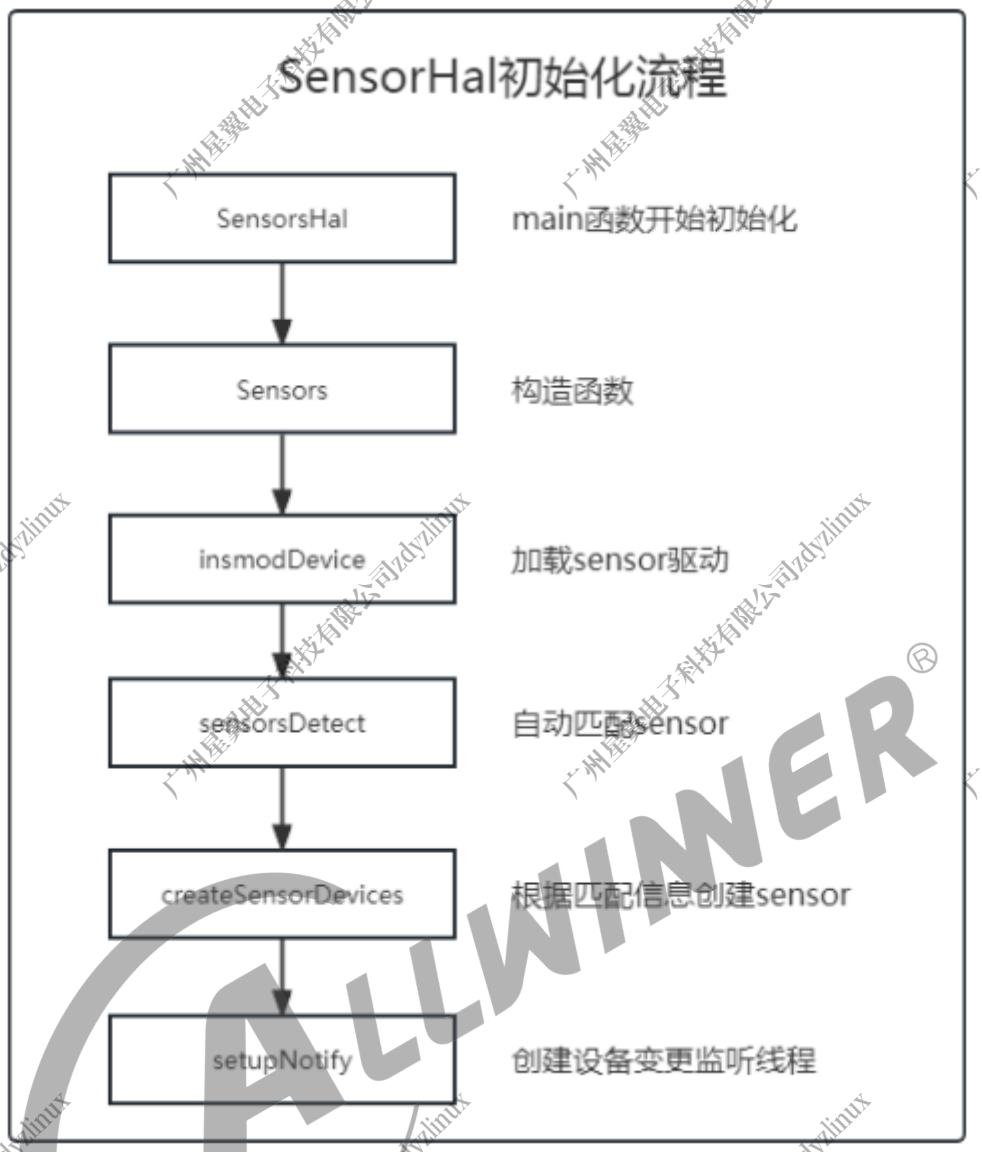

图 3-2: Sensor 框架

3.5.1.1 Sensor 驱动加载

Android 系统启动过程中，会启动 hal，Sensorhal 也会在此阶段被启动。

Sensorhal 启动过程，会加载`sensor_modules.cfg`中的驱动。

其驱动加载函数为`insmodDevice`

`sensor_modules.cfg`在方案目录中进行配置。

```
device/softwinner/xxx/xxx/input/sensor_modules.cfg
```

使用如下。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
preinsmod=init-input.ko

handle=ID_A
path=sc7a20.ko

handle=ID_A
path=mir3da.ko

path=mmc5603.ko
```

- preinsmod 为依赖模块，即 sensor 模块加载需要先加载的驱动。
- handle 为预定义的 handle 信息，当前 AIDL 版本已废弃。
- path 为模块名字，会自动寻找/vendor/lib/modules/下的对应驱动。

说明

其中，handle 在 AIDL 版本中已废弃，在 HIDL 2.0 版本中还有使用，定义在hardware/aw/sensors/aw_sensors/2.0/hal/include/sensors.h可自行查阅。

3.5.1.2 自动匹配设备

加载完驱动后，sensorhal 初始化过程会遍历 sys/class/input 目录，根据设备名自动匹配预定义的 sensor 信息，如匹配成功，则保存对应信息，用于后续 sensorhal 初始化。其函数为sensorsDetect。

预定义的 sensor 信息定义在如下路径，其详细定义参考《SensorInfo》章节。

```
hardware/aw/sensors/aw_sensors/aidl/impl/SensorInit.cpp
```

3.5.1.3 创建 sensor

获取到匹配信息后，即可自动创建设备，实现如下。

```
void Sensors::createSensorDevices() {
    for (int32_t i = 0; i < gSensorCount; i++) {
        switch (gSensorData[i].sensorInfo.type) {
            case SENSOR_TYPE_ACCELEROMETER:
                AddSensor<AccelSensor>(gSensorData[i], mNextHandle++);
                AddSensor<AccelUncalSensor>(gSensorData[i], mNextHandle++);
                break;

            case SENSOR_TYPE_LIGHT:
                AddSensor<LightSensor>(gSensorData[i], mNextHandle++);
                break;

            case SENSOR_TYPE_PROXIMITY:
                AddSensor<ProximitySensor>(gSensorData[i], mNextHandle++);
                break;

            case SENSOR_TYPE_MAGNETIC_FIELD:
                AddSensor<MagnetometerSensor>(gSensorData[i], mNextHandle++);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
                AddSensor<MagnetometerUncalSensor>(gSensorData[i], mNextHandle++);
                break;

            case SENSOR_TYPE_PRESSURE:
                AddSensor<PressureSensor>(gSensorData[i], mNextHandle++);
                break;

            case SENSOR_TYPE_AMBIENT_TEMPERATURE:
                AddSensor<AmbientTempSensor>(gSensorData[i], mNextHandle++);
                break;

            default:
                ALOGD("SensorType %d is not support!\n", gSensorData[i].sensorInfo.type);
                break;
        }
    }
}
```

部分 sensor 未进行适配，可参考上述函数进行即可。

说明

对于部分 Sensor，谷歌 GMS 要求实现 Uncal（未校准）类型，例如 AccelUncalSensor，其实现也是在 AccelSensor.cpp 里。

3.5.2 使能 Sensor

使能 sensor 是通过 Sensor 类中的 activate 方法。

```
void Sensor::activate(bool enable) {
    if (mIsEnabled != enable) {
        mIsEnabled = enable;
        setEnable(enable);
        mWaitCV.notify_all();
    }
}

void Sensor::setEnable(bool enable) {
    char buf[2];

    ALOGD("%s set enable = %d\n", mSensorName, enable);
    if (mClassPath == "" || mEnableNodeName == "") {
        ALOGE("%s classPath %s enable node %s error, please check!",
              mSensorName, mClassPath, mEnableNodeName);
    }
    int len = sprintf(buf, "%d", enable);

    setSysfsInputAttr(mClassPath, mEnableNodeName, buf, len);
    return;
}
```

activate 则会调用 setEnable 方法，setEnable 通过自动检索的 input 路径及预定义好的 enable 使能节点，自动拼接/sys/class/input/inputX/enable，并写入对应的值。

说明

有特殊需求的，可重新 setEnable 方法。

3.5.3 设置 delay

设置 delay 则是通过 batch 方法。

```
void Sensor::batch(int64_t samplingPeriodNs) {
    if (samplingPeriodNs < mSensorInfo.minDelayUs * 1000LL) {
        samplingPeriodNs = mSensorInfo.minDelayUs * 1000LL;
    } else if (samplingPeriodNs > mSensorInfo.maxDelayUs * 1000LL) {
        samplingPeriodNs = mSensorInfo.maxDelayUs * 1000LL;
    }

    mSamplingPeriodNs = samplingPeriodNs;
    // set delay
    setSamplingInterval(samplingPeriodNs);
    // Wake up the 'run' thread to check if a new event should be generated now
    mWaitCV.notify_all();
}

void Sensor::setSamplingInterval(int64_t samplingPeriodNs) {
    char buf[80];
    int len;

    ALOGD("%s set samplingPeriodNs = %" SCNi64, mSensorName, samplingPeriodNs);
    if (mClassPath == "" || mDelayNodeName == "") {
        ALOGE("%s classPath %s delay node %s error, please check!",
              mSensorName, mClassPath, mDelayNodeName);
        return;
    }
    len = sprintf(buf, "%" SCNi64, samplingPeriodNs / 1000000); /*convert ns to ms*/

    setSysfsInputAttr(mClassPath, mDelayNodeName, buf, len);
}
```

batch 会自动判断设置的 delay 是否在允许的范围内，允许则会调用 setSamplingInterval 完成 delay 的设置。setSamplingInterval 通过自动检索的 input 路径及预定义好的 delay 节点，自动拼接/sys/class/input/inputX/delay，并写入对应的值。

说明

有特殊需求的，可重新 setSamplingInterval 方法。

3.5.4 处理 event

读取 event 是通过 readEvents 方法完成。

```
std::vector<Event> Sensor::readEvents() {
    std::vector<Event> events;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    Event event;
    event.sensorHandle = mSensorInfo.sensorHandle;
    event.sensorType = mSensorInfo.type;
    memset(&event.payload, 0, sizeof(event.payload));
    readEventPayload(event.payload);
    event.timestamp = ::android::elapsedRealtimeNano();
    events.push_back(event);
    return events;
}
```

而 sensor 中要实现的则是 readEventPayload 方法，以 AccelSensor 为例，其参考处理流程如下。

```
void AccelSensor::readEventPayload(EventPayload& payload) {
    ssize_t n;
    ssize_t dataAvailable;
    input_event const* event;
    EventPayload::Vec3 vec3 = {                         //初始化默认值
        .x = 0,
        .y = 0,
        .z = -9.8,
        .status = SensorStatus::ACCURACY_HIGH,
    };

    n = mInputReader.fill(this->mInputStreamFd);        //读取数据
    if (n < 0) {
        Sensor::setCheckSelf();
        ALOGE("AceelSensor %s readEvents fill buffer failed\n", mSensorName);
        goto out;
    }
    dataAvailable = mInputReader.readEvent(&event);     //取数据
    if (dataAvailable <= 0) {
        Sensor::setCheckSelf();
        ALOGE("sensor %s no sensor data available\n", mSensorName);
        goto out;
    }

    while (dataAvailable) {
        int type = event->type;

        if ((type == EV_ABS) || (type == EV_REL) || (type == EV_KEY)) {
            processEvent(event->code, event->value);                //针对取到的3轴数据进行处理
        } else if (type == EV_SYN) {                                //sync时表示一次完整完成
            if (reviseEvent()) {
                break;
            }
        }
        dataAvailable = mInputReader.readEvent(&event);             //继续读取数据
    }

out:
    vec3.x = this->mCurEventData[0];
    vec3.y = this->mCurEventData[1];
    vec3.z = this->mCurEventData[2];
    payload.set<EventPayload::Tag::vec3>(vec3);                     //返回数据
}
```

其中，processEvent 是通常每个 sensor 自行实现，对读取的原始数据进行处理转换的地方。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 支持 Sensorhal

4.1 编译

在机器上增加 sensorhal 支持，需要在方案中将 sensorhal 编译进去。

```
PRODUCT_PACKAGES += android.hardware.sensors-service.aw
```

在 Android 12 及之前版本，使用 HIDL 2.0 版本。

```
PRODUCT_PACKAGES += android.hardware.sensors@2.0-service
```

4.2 添加 feature

支持一款新 sensor 时，需要添加对应的 feature，否则可能会造成一些 app 无法获取 sensor，或 GMS 测试失败。

android 的 feature 文件以及支持的很好了，我们需要做的是将需要的 feature 文件复制到指定的目录。

1. Android 原生的 feature 文件存放路径。

```
frameworks/native/data/etc
```

里面的 feature 文件。

```
android.hardware.sensor.accelerometer_limited_axes_uncalibrated.xml
android.hardware.sensor.accelerometer_limited_axes.xml
android.hardware.sensor.accelerometer.xml
android.hardware.sensor.ambient_temperature.xml
android.hardware.sensor.assist.xml
android.hardware.sensor.barometer.xml
android.hardware.sensor.compass.xml
android.hardware.sensor.dynamic.head_tracker.xml
android.hardware.sensor.gyroscope_limited_axes_uncalibrated.xml
android.hardware.sensor.gyroscope_limited_axes.xml
android.hardware.sensor.gyroscope.xml
android.hardware.sensor.heading.xml
android.hardware.sensor.heartrate.ecg.xml
android.hardware.sensor.heartrate.fitness.xml
android.hardware.sensor.heartrate.xml
android.hardware.sensor.hifi_sensors.xml
android.hardware.sensor.hinge_angle.xml
android.hardware.sensor.light.xml
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
android.hardware.sensor.proximity.xml
android.hardware.sensor.relative_humidity.xml
android.hardware.sensor.stepcounter.xml
android.hardware.sensor.stepdetector.xml
```

2. 方案中复制需要的 feature 文件到指定路径。

```
PRODUCT_COPY_FILES += \
    frameworks/native/data/etc/android.hardware.sensor.accelerometer.xml:$(TARGET_COPY_OUT_VENDOR)/etc/permissions/android.hardware.sensor.accelerometer.xml \
    frameworks/native/data/etc/android.hardware.sensor.compass.xml:$(TARGET_COPY_OUT_VENDOR)/etc/permissions/android.hardware.sensor.compass.xml \
```

图 4-1: sensor_feature

如图所示，当前版型支持了 accelsensor（重力感应）和 compass（磁力计）。

注意在 Android 12 及之前版本，使用的是自定义的 feature 文件，其名字为 sensor_feature.xml，在 Android13 已删除。

⚠ 注意

不支持某类 sensor 时，应将对应的 sensorfeature 删除，避免出现相关问题。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 适配 Sensor

适配一款新 Sensor 时，通常有 2 种情况。

1. 该 sensor 类型在 Sensorhal 已有默认代码实现。
2. 该 sensor 类型在 Sensorhal 从未适配。

下面针对 2 种情况进行描述

说明

这里适配的 sensor 指驱动已适配完成，能正常上报数据，提供 enable，delay 节点。

5.1 已实现 Sensor

针对已实现的 sensor，以 AccelSensor（重力感应 gsensor）为例。其实现如下。

5.1.1 加载驱动

参考 SensorHal 工作流程-Sensor 驱动加载，在对应方案目录中的 sensor_modules.cfg，添加需要加载的驱动信息。

5.1.2 添加 Sensor 信息

参考 SensorInfo 章节，在 SensorInit.cpp 中添加对应的 Sensor 信息。

说明

在 hidl 版本，则是在 sensorDetect.cpp 中添加对应信息。

5.1.3 数据处理

AccelSensor 通常需要对数据进行转换，当前 Accel 使用的数据转换为如下格式。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
x = raw_x X priData X revertX
```

其中，priData 为地球标准重力/pri 数据，revertX 指是否需要对 X 进行翻转，该数据通过方向适配而来，会在后面描述。

```
priData = GRAVITY_EARTH / sensorData.priData;
```

5.1.4 AccelSensor 方向适配

AccelSensor 通过用于自动旋转，而贴片的方向是不确定的，因此很多时候还需要对 AccelSensor 的方向进行适配。

5.1.4.1 方向配置文件

在方案目录，还存在如下配置文件。用于记录预定义的方向。

```
device/softwinner/xxx/xxx/input/gsensor.cfg
```

这里用于存放调试好的方向值，每个设备都有五项值，如下所示。

字段	含义
gsensor_name	Gsensor 名称，必须与驱动中设备名相同
gsensor_direct_x	Gsensor x 轴的方向，当定义成 true 时，x 轴取正值，当定义为 false 时，x 轴取负值
gsensor_direct_y	Gsensor y 轴的方向，当定义成 true 时，y 轴取正值，当定义为 false 时，y 轴取负值
gsensor_direct_z	Gsensor z 轴的方向，当定义成 true 时，z 轴取正值，当定义为 false 时，z 轴取负值
gsensor_xy_revert	XY 轴对调，当设为 TRUE 时，x 轴变为原来 y 轴

参考格式例子如下。

```
;--------------------------
;name:sc7a20
;--------------------------
gsensor_name = sc7a20
gsensor_direct_x = false
gsensor_direct_y = false
gsensor_direct_z = false
gsensor_xy_revert = true
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.1.4.2 方向配置

方向适配主要有如下配置，针对不同的场景及需求。

1. gsensor.cfg：存在配置好的方向信息。
2. ro.vendor.sf.rotation：Sensor 的旋转角度，可配置为 [0, 90, 180, 270]，通常配置为 0。
3. ro.vendor.gsi_gsen_rotation：针对 GSI 场景下的旋转角度，可配置为 [0, 90, 180, 270]，无 GSI 需求可忽略。

5.1.4.3 方向适配

方向的适配存在几种情况。

1. 有 GSI 需求，且存在 GSI 和量产固件方向不一致的情况。
2. 无 GSI 需求，忽略 ro.vendor.gsi_gsen_rotation 配置即可。

1. 配置前的准备

配置步骤及配置前的准备如下。

1. 已根据《Android_rotation_使用指南》进行了屏幕方向的适配。
2. 使用模组前，请确认 gsensor.cfg 文件中是否已经存在模组的方向配置项，若没有，请添加模组的方向配置项，并将各项默认配置为 0。
3. 设置 ro.vendor.sf.rotation 和 ro.vendor.gsi_gsen_rotation 为 0。

2. cfg 配置

gsensor.cfg 该文件存放在机器的/vendor/etc 目录下，当发现方向不正确时，按照如下步骤进行调试。

Gsensor 方向调试说明：

假定机器的长轴为 X 轴，短轴为 Y 轴，垂直方向为 Z 轴。在 android 系统下，可通过专门的 apk 获取 gsensor 的三轴数据，或者通过 dumpsys sensorservice 来获取三轴数据。

首先调试 Z 轴：

第一步观察现象：

机器平放屏幕朝上，获取 gsensor 的轴数据，平放时，z 轴数据正常是接近 9.8(G 的值)，如果 z 轴数据约为-9.8，则 z 轴反了，否则 z 轴正确。

第二步修改 Z 轴为正确方向:

版权所有 © 珠海全志科技股份有限公司。保留一切权利

从 gsensor.cfg 配置中找到对应 gsensor 的配置。如果此时该方向 Z 轴向量（gsnesor_direct_z）的值为 false，则需要修改为 true；当为 true，则需要修改为 false。通过 adb shell 将修改后的 gsnesor.cfg 文件 push 到/vendor/etc 下，重启机器，按第一步观察现象。

其次调试 x、y 轴：

第一步观察现象：

首先将机器旋转到 android 显示的 0 度方向，机器垂直于水平面放置，获取 x、y 轴的数据，此时正常情况下，x 轴数据接近为 0，y 轴数据接近 9.8(G 的值)，z 轴接近为 0。如果此时 x 轴接近 (-)9.8 的值，y 轴数据接近为 0，则 x、y 轴需交换。

第二步交换 X，Y 方向

从 gsensor.cfg 配置中找到对应 gsensor 的配置。如果此时该 X，Y 轴互换向量（gsensor_xy_revert）的值为 false，则需要修改为 true，当为 true，则需要修改为 false。通过 adb remount 及 adb shell 将修改后的 gsnesor.cfg 文件 push 到/vendor/etc 下，重启机器，按第一步观察现象。

再次调试 X，Y 轴方向：

第一步观察现象：

首先将机器旋转到 android 显示的 0 度方向，机器垂直于水平面放置，获取 y 轴的数据，此时如果 y 轴数据接近正值的 G 值，则 y 轴方向正确，如果接近负值的 G 值，则 y 轴方向相反。

再将机器旋转到 android 显示的 90 度方向，机器垂直于水平面放置，获取 x 轴数据，此时如果 x 轴数据接近正值的 G 值，则 x 轴方向正确，如果接近负值的 G 值，则 x 轴方向相反。

第二步修改 X，Y 轴方向：

当需要修改 X，Y 轴方向时，当只有长轴方向相反或者是只有短轴方向相反时，则只修改方向不正确的一个轴，当两个方向都相反时，则同时修改 X 与 Y 轴方向向量。找到当前使用模组的方向向量（根据模组的名称）。

若长轴方向相反，如果此时该方向 X 轴向量（gsnesor_direct_x）的值为 false，则需要修改为 true，当为 true，则需要修改为 false。

若短轴方向相反，如果此时该方向 Y 轴向量（gsnesor_direct_y）的值为 false，则需要修改为 true，当为 true，则需要修改为 false。

通过 adb shell 将修改后的 gsnesor.cfg 文件 push 到/vendor/etc 下，重启机器，按第一步观察现象。

3. gsi 配置

基于量产固件配置完成 cfg 后且显示正常后，烧录 GSI，然后观察屏幕旋转方向，如旋转正常，则 ro.vendor.gsi_gsen_rotation 配置为 0 即可。

如果旋转异常，可参考如下进行配置。

1. 方向翻转 180°，则配置 ro.vendor.gsi_gsen_rotation=180。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 如果方向向左或者向右旋转了 90°，则可尝试配置 ro.vendor.gsi_gsen_rotation 为 90 或者 270 度进行尝试。

5.2 未定义的 sensor 类型

适配一个 SensorHal 中不存在的 sensor，可参考如下步骤。

1. 新建一个 sensor 类型的 cpp，名字可参考 AccelSensor。
2. 实现构造函数
3. 重写 processEvent，readEventPayload 方法，其他方法则看需求重写。
4. 参考已实现 Sensor 适配章节进行适配即可。

广州星翼电子科技有限公司zdyzlinux

6 支持 Sensor 校准

6.1 Sensor 校准原理

传感器在制造、使用和环境中会产生误差。（如加速度计静态时 Z 轴理论值应为 +9.81 m/s^2，实际常存在 ±0.5 m/s^2 误差）

Sensor 校准是指将传感器的输出值与已知精度更高的标准仪器或参考值进行比较，通过调整或修正传感器的输出，以减少其测量误差的过程。

校准可以消除由环境变化、元器件老化、制造差异等因素引起的误差，确保传感器输出更接近真实值。

说明

因校准数据保存在设备的 nv 分区（私有/安全分区），需注意刷机时不能使用全盘擦除升级，否则会造成数据丢失。

6.2 支持的传感器类型

6.2.1 加速度传感器，重力传感器，磁力传感器

对于这三类传感器，校准偏移值有三个，分别是 xBias，yBias，zBias。其都存在对应的 Uncal 类型的传感器，而 Uncal 类型的偏差值，例如 AccelUncalSensor，在

AccelSensor 执行校准的时候会自动进行赋值。

同样的，在进行 AccelUncalSensor 的校准时，AccelSensor 也会自动执行校准。

注意

这三类传感器和其 Uncal 类型的 sensorName 必须相同，否则校准数据无法同步。

6.2.2 光线传感器，距离传感器，压力传感器，温度传感器

对于这四类传感器，校准偏移值只有一个，分别对应 lightBias，proxiBias，pressureBias，tempBias。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.3 Sensorhal 为一个新类型的 sensor 支持校准

前置步骤可参考适配 Sensor 章节。

下面以 AccelSensor 为例，说明如何针对 AccelSensor 增加该校准功能。

```
//hardware/aw/sensors/aw_sensors/aidl/impl/AccelSensor.cpp
void AccelSensor::readEventPayload(EventPayload& payload) {
out:
    vec3.x = this->mCurEventData[0] + mCalibrationData.xyz.xBias;
    vec3.y = this->mCurEventData[1] + mCalibrationData.xyz.yBias;
    vec3.z = this->mCurEventData[2] + mCalibrationData.xyz.zBias;
    payload.set<EventPayload::Tag::vec3>(vec3);
}

void AccelUncalSensor::readEventPayload(EventPayload& payload) {
    EventPayload::Uncal uncal = {
        .x = 0,
        .y = 0,
        .z = -9.8,
        .xBias = mCalibrationData.xyz.xBias,
        .yBias = mCalibrationData.xyz.yBias,
        .zBias = mCalibrationData.xyz.zBias,
    };
out:
    uncal.x = this->mCurEventData[0];
    uncal.y = this->mCurEventData[1];
    uncal.z = this->mCurEventData[2];
    payload.set<EventPayload::Tag::uncal>(uncal);
}
```

1. 在 AccelSensor::readEventPayload 函数中将 mCalibrationData.xyz 中的偏差值 xBias、yBias、zBias 加到原始传感器读数上。

2. 如有定义 Uncal 类型的传感器，需在 AccelUncalSensor::readEventPayload 函数中将 mCalibrationData.xyz 中的偏差值 xBias、yBias、zBias 赋值给 uncal 变量。

说明

新类型 sensor 的偏差值变量名可在 Sensor.h 文件 CalibrationData 结构体中定义。

```
hardware/aw/sensors/aw_sensors/aidl/impl/include/Sensor.h
```

```
struct CalibrationData {
    struct xyzCalibration {
        float xBias = 0.0f;
        float yBias = 0.0f;
        float zBias = 0.0f;
    } xyz;

    struct LightCalibration {
        float lightBias = 0.0f;
    } light;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    struct PressureCalibration {
        float pressureBias = 0.0f;
    } pressure;

    struct TempCalibration {
        float tempBias = 0.0f;
    } temp;

    struct ProximityCalibration {
        float proxiBias = 0.0f;
    } proximity;
};
```

6.4 校准功能配置开关

可通过配置属性`persist.sys.awsensorscalibration.enable`为 true 来默认打开传感器校准功能；功能默认关闭。

如果需要默认打开传感器校准功能，可参考下列配置步骤。

1. 在对应的 common 仓库下，找到如下文件

common 仓库路径为：

```
device/softwinner/commom
```

则对应的配置文件为 common.mk。

说明

如果没有该文件，可自行创建对应文件。

2. 默认使能支持校准。

```
# Set default value for AwSensorsCalibration enable
PRODUCT_PROPERTY_OVERRIDES += \
    persist.sys.awsensorscalibration.enable=true
```

如上，将`persist.sys.awsensorscalibration.enable`配置为 true，即可默认启用校准功能。

说明

当设备开机时，就会自动读取保存在 nv 分区（私有/安全分区）的校准值，将校准值加在传感器读取的原始数据上，为设备上所有传感器自动执行一次校准。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.5 Android 校准功能支持

6.5.1 有哪些接口

```
vendor/aw/public/framework/base/ex-interface/core/java/com/softwinner/awsensorcalibration/
    AwSensorsCalibrationManager.java
```

此.java 文件位于以上路径，有四个接口：

6.5.1.1 setCalibrationData

- 作用：设置指定传感器名称为 sensorName 的的校准数据 data 到 secure/private，同时进行校准
- 参数：
- sensorName : 传感器名称（如 “Msa_3-axis_Accel”，“stk3x1x_AmbLightS” 等）
- data : 校准参数字符串
- 返回值：boolean
- true: success
- false：fail
- 异常：如果 sensorName 为 null，抛出 IllegalArgumentException

6.5.1.2 getCalibrationData

- 作用：获取指定传感器名称为 sensorName 的校准数据 data
- 参数：
- sensorName : 传感器名称
- 返回值：String
- true: 校准数据字符串
- false：null
- 异常：如果 sensorName 为 null，抛出 IllegalArgumentException

6.5.1.3 clearCalibrationTmpData

- 作用：清除传感器名称为 sensorName 的 HAL 层中的校准临时数据，等同于不做校准。
- 参数：
- sensorName : 传感器名称

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回值：boolean
- true: success
- false：fail
- 异常：如果 sensorName 为 null，抛出 IllegalArgumentException

6.5.1.4 clearCalibrationSstData

- 作用：清除名称为 sensorName 的 secure/private 中的校准数据 data
- 参数：
- sensorName : 传感器名称
- 返回值：boolean
- true: success
- false：fail
- 异常：如果 sensorName 为 null，抛出 IllegalArgumentException

说明

如需修改某种传感器的名称，可在 hardware/aw/sensors/aw_sensors/aidl/impl/SensorInit.cpp 中查找修改。

6.5.2 写入数据格式要求

6.5.2.1 加速度传感器，重力传感器，磁力传感器

```
xBias=xxx,yBias=xxx,zBias=xxx
```

6.5.2.2 光线传感器

```
lightBias=xxx
```

6.5.2.3 温度传感器

```
tempBias=xxx
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.5.2.4 压力传感器

```
pressureBias=xxx
```

6.5.2.5 距离传感器

```
proxiBias=xxx
```

具体写入数据格式示例可参考 APP 适配流程章节：使用 demo 示例

6.5.3 APP 适配流程

6.5.3.1 权限声明和参数配置

1. 声明为系统应用，使用平台 API 接口 platform_apis: true, 并使用系统签名 certificate: “platform”

```
android_app {
    certificate: "platform"    //使用系统签名
    platform_apis: true,
    manifest: "app/src/main/AndroidManifest.xml",
    privileged: true,  // 标记为系统应用
}
```

2.Manifest 文件中添加系统权限配置。

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.softwinner.sensorcalibrationdemo">

    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>

</manifest>
```

6.5.3.2 如何获取实例化对象

```
import com.softwinner.awsensorcalibration.AwSensorsCalibrationManager;

AwSensorsCalibrationManager manager = new AwSensorsCalibrationManager(context);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.5.3.3 使用 demo 示例

```
package com.softwinner.sensorcalibrationdemo;

import android.content.Context;
import android.hardware.Sensor;
import android.hardware.SensorManager;
import android.util.Log;

import com.softwinner.awsensorcalibration.AwSensorsCalibrationManager;

import java.util.List;

public class SensorCalibrationTest {
    private static final String TAG = "SensorCalibrationTest";

    public void testAllInterfaces(Context context) {
        AwSensorsCalibrationManager manager = new AwSensorsCalibrationManager(context);
        SensorManager sensorManager = (SensorManager)context.getSystemService(Context.
    SENSOR_SERVICE);
        Log.d(TAG, "成功创建 AwSensorsCalibrationManager 实例");
        List<Sensor> sensors =sensorManager.getSensorList(Sensor.TYPE_ALL);

        if (sensors.isEmpty()) {
            Log.w(TAG,"没有找到传感器");
            return;
        }


        for (Sensor sensor: sensors){
//              String sensorName = sensor.getName();
            String sensorName = "Msa_3-axis_Accel";
            Log.d(TAG, "测试传感器：" + sensorName);

            testSensorInterfaces(manager, sensorName);
        }
    }

     private void testSensorInterfaces(AwSensorsCalibrationManager manager, String
    sensorName){

        // 测试1: 设置数据
           try {
               boolean success = manager.setCalibrationData(
                       sensorName,
                       "xBias=0,yBias=0.5,zBias=5.0"
               );
               Log.d(TAG, "setCalibrationData('" + sensorName + "') 结果: " + success);
           } catch (Exception e) {
               Log.e(TAG, "setCalibrationData('" + sensorName + "') 失败", e);
           }

        // 测试2: 获取数据
        try {
            String result = manager.getCalibrationData(sensorName);
            Log.d(TAG, "getCalibrationData('" + sensorName + "') 结果: " + result);
        } catch (Exception e) {
            Log.e(TAG, "getCalibrationData('" + sensorName + "') 失败", e);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        }

//          // 测试3: 清理临时数据
//          try {
//              boolean success = manager.clearCalibrationTmpData(sensorName);
//              Log.d(TAG, "clearCalibrationTmpData('" + sensorName + "') 结果: " + success);
//          } catch (Exception e) {
//              Log.e(TAG, "clearCalibrationTmpData('" + sensorName + "') 失败", e);
//          }
//
//          // 测试4: 清理SST数据
//          try {
//              boolean success = manager.clearCalibrationSstData(sensorName);
//              Log.d(TAG, "clearCalibrationSstData('" + sensorName + "') 结果: " + success);
//          } catch (Exception e) {
//              Log.e(TAG, "clearCalibrationSstData('" + sensorName + "') 失败", e);
//          }
    }
}
```

最后在服务器上进行编译。进入 app 的文件目录，通过 m SensorCalibrationDemo 编译，再 install 在机器里。

6.5.4 如何 Debug

6.5.4.1 确定数据是否写入

通过 sst_test_v2 -l 查看校准数据 data 有没有写入里面可以看到每个 sensorName 对应的校准数据 data

```
a537-pro:/ # sst_test_v2 -l

[secure storage]
Msa_3-axis_Accel = xBias=0,yBias=0.5,zBias=5.0
stk3x1x_AmbLightS = lightBias=10.0
stk3x1x_ProxiS = proxiBias=1.0

[private space]
no key!!
```

若 data 没有写入，通过日志打印 logcat | grep sensorhal -i 对比以下路径代码分析哪里出错

```
hardware/aw/sensors
```

```
logcat | grep sensorhal -i
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
a537-pro:/ # logcat | grep sensorhal -i
09-12 18:23:25.766   581   581 D SensorHal: Sensor Aidl Hal init Version AIDL-V2.0.0
09-12 18:23:25.767   581   581 D SensorHal: buf:preinsmod=init-input.ko
09-12 18:23:25.767   581   581 D SensorHal:
09-12 18:23:25.778   581   581 D SensorHal: buf:
09-12 18:23:25.778   581   581 D SensorHal:
09-12 18:23:25.778   581   581 D SensorHal: buf:path=msa.ko
09-12 18:23:25.778   581   581 D SensorHal:
09-12 18:23:25.842   581   581 D SensorHal: buf:path=stk3x1x.ko
09-12 18:23:25.842   581   581 D SensorHal:
09-12 18:23:25.914   581   581 D SensorHal: deviceName : stk3x1x_ls
09-12 18:23:25.914   581   581 D SensorHal: classPath = /sys/class/input/input9 gSensorData[gSensorCount].classPath = /sys/class/input/input9
09-12 18:23:25.915   581   581 D SensorHal: deviceName : audiocodec Headphones
09-12 18:23:25.916   581   581 D SensorHal: deviceName : axp2202-pek
09-12 18:23:25.918   581   581 D SensorHal: deviceName : stk3x1x_ps
09-12 18:23:25.918   581   581 D SensorHal: classPath = /sys/class/input/input10 gSensorData[gSensorCount].classPath = /sys/class/input/input10
09-12 18:23:25.918   581   581 D SensorHal: deviceName : msa
09-12 18:23:25.918   581   581 D SensorHal: classPath = /sys/class/input/input8 gSensorData[gSensorCount].classPath = /sys/class/input/input8
09-12 18:23:25.919   581   581 D SensorHal: deviceName : sunxi-keyboard
09-12 18:23:26.087   581   581 E SensorHal: couldn't find 'stk3x1x_ls' input device
09-12 18:23:26.087   581   581 E SensorHal: set EVIOCSCLOCKID failed !
09-12 18:23:26.088   581   581 D SensorHal: LightSensor Construct!
09-12 18:23:26.088   581   581 D SensorHal: ProximitySensor Construct!
09-12 18:23:26.151   581   581 D SensorHal: AccelSensor Construct!
09-12 18:23:26.203   581   581 D SensorHal: AccelSensor Construct!
09-12 18:23:26.204   581   581 D SensorHal: AccelUncalSensor Construct!
09-12 18:23:26.204   581   581 D SensorHal: readSstData: reading key=stk3x1x_AmbLightS
09-12 18:23:26.209   581   581 D SensorHal: readSstData: successfully read 16 bytes: lightBias=4096.0
09-12 18:23:26.209   581   581 D SensorHal: Successfully read SST data: lightBias=4096.0
09-12 18:23:26.209   581   581 D SensorHal: [stk3x1x_ls] received calibration data: lightBias=4096.0
09-12 18:23:26.209   581   581 D SensorHal: readSstData: reading key=stk3x1x_ProxiS
09-12 18:23:26.213   581   581 D SensorHal: readSstData: successfully read 12 bytes: proxiBias=3096.0
09-12 18:23:26.213   581   581 D SensorHal: Successfully read SST data: proxiBias=30
09-12 18:23:26.213   581   581 D SensorHal: [stk3x1x_ps] received calibration data: proxiBias=30
09-12 18:23:26.213   581   581 D SensorHal: readSstData: reading key=Msa_3-axis_Accel
09-12 18:23:26.222   581   581 D SensorHal: readSstData: successfully read 26 bytes: xBias=0,yBias=0.5,zBias=88
09-12 18:23:26.222   581   581 D SensorHal: Successfully read SST data: xBias=0,yBias=0.5,zBias=88
09-12 18:23:26.222   581   581 D SensorHal: [msa] received calibration data: xBias=0,yBias=0.5,zBias=88
09-12 18:23:26.222   581   581 D SensorHal: [msa] synced calibration data to another sensor
09-12 18:23:26.222   581   581 D SensorHal: readSstData: reading key=Msa_3-axis_Accel
09-12 18:23:26.223   581   581 D SensorHal: readSstData: successfully read 26 bytes: xBias=0,yBias=0.5,zBias=88
09-12 18:23:26.223   581   581 D SensorHal: Successfully read SST data: xBias=0,yBias=0.5,zBias=88
09-12 18:23:26.223   581   581 D SensorHal: [msa] received calibration data: xBias=0,yBias=0.5,zBias=88
09-12 18:23:26.223   581   581 D SensorHal: [msa] synced calibration data to another sensor
09-12 18:23:26.224   581   581 D SensorHal: NotifyThread start
09-12 18:23:26.225   581   581 D SensorHal: CalibrationThread start
09-12 18:23:39.608   581   581 D SensorHal: initialize
09-12 18:23:39.608   581   581 D SensorHal: deleteEventFlag
09-12 18:24:07.348   581   581 D SensorHal: msa set samplingPeriodNs = 66667000
09-12 18:24:07.350   581   581 D SensorHal: AccelSensor handle(3) setEnable enable(1) anotherSensorEnable (0)
09-12 18:24:07.350   581   581 D SensorHal: msa set enable = 1
09-12 18:29:00.730   581   581 D SensorHal: AccelSensor handle(3) setEnable enable(0) anotherSensorEnable (0)
09-12 18:29:00.730   581   581 D SensorHal: msa set enable = 0
```

图 6-1: sensorhal_logcat 打印

6.5.4.2 dumpsys sensorservice 的使用

调用 setCalibrationData 方法后需确定校准数据是否已经生效，在 cmd 通过 adb shell dumpsys sensorservice 查看

Recent Sensor events 显示了机器中各传感器的最近发生的事件，通过观察每个事件的值进行判断。

如 sensorname 为 Msa_3-axis_Accel 的传感器，机器平放时为 0,0,9.8 应用校准数据之后事件值有变化才为生效

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Recent Sensor events:
Msa_3-axis_Accel: last 50 events
         1 (ts=363.105534547, wall=18:28:58.104) 0.10, 0.48, 97.70,
         2 (ts=363.169330464, wall=18:28:58.168) 0.04, 0.48, 97.74,
         3 (ts=363.233246381, wall=18:28:58.232) 0.04, 0.48, 97.70,
         4 (ts=363.297144172, wall=18:28:58.296) 0.06, 0.46, 97.70,
         5 (ts=363.361412756, wall=18:28:58.360) 0.06, 0.46, 97.74,
         6 (ts=363.361534798, wall=18:28:58.360) 0.06, 0.46, 97.74,
         7 (ts=363.428753673, wall=18:28:58.427) 0.04, 0.48, 97.70,
         8 (ts=363.488785798, wall=18:28:58.487) 0.06, 0.50, 97.72,
         9 (ts=363.553674714, wall=18:28:58.552) 0.04, 0.48, 97.72,
        10 (ts=363.616317298, wall=18:28:58.615) 0.00, 0.48, 97.72,
        11 (ts=363.681045673, wall=18:28:58.680) 0.08, 0.46, 97.72,
        12 (ts=363.681142631, wall=18:28:58.680) 0.08, 0.46, 97.70,
        13 (ts=363.744708339, wall=18:28:58.743) 0.06, 0.48, 97.76,
        14 (ts=363.808953423, wall=18:28:58.808) 0.02, 0.48, 97.72,
        15 (ts=363.872722548, wall=18:28:58.871) 0.04, 0.48, 97.72,
        16 (ts=363.937096839, wall=18:28:58.936) 0.00, 0.48, 97.74,
        17 (ts=364.000453589, wall=18:28:58.999) 0.02, 0.48, 97.76,
        18 (ts=364.000578381, wall=18:28:58.999) 0.02, 0.48, 97.76,
        19 (ts=364.064843256, wall=18:28:59.063) 0.04, 0.46, 97.68,
        20 (ts=364.129438215, wall=18:28:59.128) 0.06, 0.48, 97.72,
        21 (ts=364.196430506, wall=18:28:59.195) 0.06, 0.48, 97.74,
        22 (ts=364.256702965, wall=18:28:59.255) 0.02, 0.48, 97.68,
        23 (ts=364.321097090, wall=18:28:59.320) 0.06, 0.44, 97.68,
        24 (ts=364.321689298, wall=18:28:59.320) 0.06, 0.44, 97.76,
        25 (ts=364.384879465, wall=18:28:59.384) 0.00, 0.48, 97.66,
        26 (ts=364.449069048, wall=18:28:59.448) 0.04, 0.50, 97.68,
        27 (ts=364.512785590, wall=18:28:59.511) 0.06, 0.46, 97.72,
        28 (ts=364.576636340, wall=18:28:59.575) 0.04, 0.48, 97.72,
        29 (ts=364.577157881, wall=18:28:59.576) 0.04, 0.48, 97.70,
        30 (ts=364.640546756, wall=18:28:59.639) 0.02, 0.44, 97.70,
        31 (ts=364.705522756, wall=18:28:59.704) 0.04, 0.44, 97.72,
        32 (ts=364.769597340, wall=18:28:59.768) 0.04, 0.46, 97.68,
        33 (ts=364.833592340, wall=18:28:59.832) 0.00, 0.48, 97.70,
        34 (ts=364.897566548, wall=18:28:59.896) 0.06, 0.48, 97.70,
        35 (ts=364.898112132, wall=18:28:59.897) 0.06, 0.50, 97.78,
        36 (ts=364.968450673, wall=18:28:59.967) 0.02, 0.44, 97.78,
        37 (ts=365.026438132, wall=18:29:00.025) 0.04, 0.48, 97.80,
        38 (ts=365.088410507, wall=18:29:00.087) 0.06, 0.44, 97.74,
        39 (ts=365.152471132, wall=18:29:00.151) 0.00, 0.44, 97.68,
        40 (ts=365.152807048, wall=18:29:00.151) 0.00, 0.44, 97.68,
        41 (ts=365.216538923, wall=18:29:00.215) 0.04, 0.46, 97.66,
        42 (ts=365.281311965, wall=18:29:00.280) 0.06, 0.48, 97.68,
        43 (ts=365.345557007, wall=18:29:00.344) 0.06, 0.46, 97.68,
        44 (ts=365.409481465, wall=18:29:00.408) 0.02, 0.46, 97.72,
        45 (ts=365.473475340, wall=18:29:00.472) 0.04, 0.48, 97.72,
        46 (ts=365.474004382, wall=18:29:00.473) 0.04, 0.48, 97.76,
        47 (ts=365.536953132, wall=18:29:00.536) 0.10, 0.48, 97.76,
        48 (ts=365.600551590, wall=18:29:00.599) 0.06, 0.46, 97.74,
        49 (ts=365.665022424, wall=18:29:00.664) 0.02, 0.48, 97.70,
        50 (ts=365.728295590, wall=18:29:00.727) 0.04, 0.48, 97.64,
```

图 6-2: 最近 sensor 事件

广州星翼电子科技有限公司zdyzlinux

7 抬起唤醒开发

7.1 Gsensor 抬起唤醒原理

Gsensor 抬起唤醒的原理，是基于 Gsensor 在设备端休眠时，可以检测到重力加速度值变化，达到一定阈值后，通过中断触发的方式，来通知设备端唤醒；而设备端唤醒后，可通过获取 Gsensor 重力值变化，来判断是否唤醒 Android 系统并亮屏，从而实现抬起唤醒功能；其大致原理图如下。

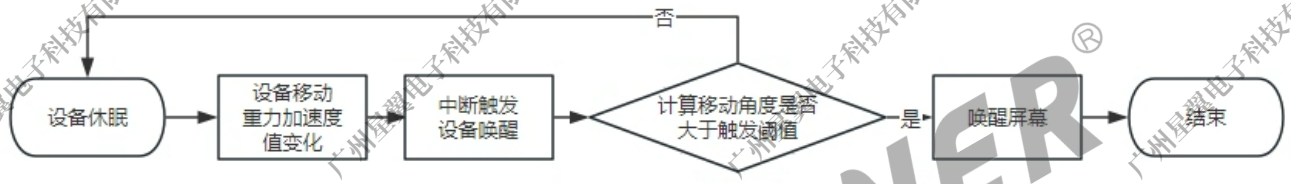

图 7-1: Gsensor 抬起唤醒流程图

7.2 功能所需硬件要求

基于 Gsensor 抬起唤醒原理，针对 Gsensor 抬起唤醒所需硬件要求如下：

- Gsensor 支持抬起唤醒，可检测到重力加速度值变化并触发中断；
- Gsensor 需有中断，且中断脚需连接到 CPUs 域；
- Gsensor 的 VCC 在休眠时需不掉电；
- 其他 Gsensor 对于抬起唤醒功能的外围硬件要求。

故需要该功能时，请先和 Gsensor 供应商确认是否支持，并做好硬件原理图 check。

7.3 MSA321 Gsensor 抬起唤醒

以 MSA321 GSENSOR 为例，说明如何针对 Gsensor 增加该功能。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7.3.1 设备树配置

MSA sensor 设备树配置如下。

```
&twi5 {
        msa {
                compatible = "allwinner,msa";
                reg = <0x62>;
                device_type = "gsensor";
                status = "okay";
                wakeup-source;
                gsensor_twi_id = <0x5>;
                gsensor_twi_addr = <0x62>;
                gsensor_int = <&r_pio PL 4 (GPIO_ACTIVE_LOW | GPIO_PULL_UP)>;
                gsensor-supply = <&reg_aldo3>;
                gsensor_vcc_io_val = <3300>;
        };
```

其中，关键配置说明如下：

- gsensor_int：中断脚配置，主要是 PL4 引脚；
- gsensor-supply：电源配置，采用的是 aldo3，休眠时不会掉电。

技巧

中断脚如在 CPUs 域，则配置应是 &r_pio，而非 pio。

7.3.2 驱动适配

驱动适配主要分为下列 5 个步骤，其说明如下：

1. 中断申请：申请中断并注册中断唤醒源；
2. 注册新设备：注册一个新的 input 设备来上报唤醒事件；
3. 设备节点创建：创建一个 sysfs 节点路径来开启和关闭抬起唤醒功能；
4. 中断处理：在中断处理函数里计算抬起角度并决策是否上报唤醒事件；
5. 休眠唤醒处理：在抬起唤醒功能开启时做特殊处理，避免休眠掉电。

其具体的适配流程见下列章节。

7.3.2.1 中断申请

中断申请流程示例。

```
irq_number = gpio_to_irq(gsensor_info.int_number);
if (irq_number < 0) {
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        MI_ERR("get irq number err");
        goto err_remove_sysfs_group;
}
err = request_threaded_irq(irq_number, NULL, msa_irq_handler,
                           IRQF_TRIGGER_FALLING | IRQF_NO_SUSPEND | IRQF_ONESHOT,
                           MSA_GESTURE_DEVICE_NAME, msa_handle);
if (err < 0) {
        printk("Failed to request gpio irq \n");
        goto err_request_irq;
}

input_set_int_enable(&(gsensor_info.input_type), 0);
device_init_wakeup(&(msa_gestrue_idev->dev), 1);
dev_pm_set_wake_irq(&msa_gestrue_idev->dev, irq_number);
```

主要步骤如下：

1. 获取中断号，这里通过 gpio_to_irq 获；
2. 申请中断线程，使用 request_threaded_irq 申请；
3. 关闭中断，原因是中断申请后是默认使能的，需要先关闭，使用时打开；
4. 注册成唤醒设备，通过 device_init_wakeup 实现；
5. 将中断申请为中断唤醒源，通过 dev_pm_set_wake_irq。

⚠ 注意

申请必须申请中断线程，原因是中断唤醒通常需要等待调度，必须使用中断线程才可以调度。

7.3.2.2 注册新设备

为避免与 gsensor 设备冲突，通常申请一个新的 input 设备来上报唤醒事件，下为示例：

```
msa_gestrue_idev = input_allocate_device();
if (!msa_gestrue_idev) {
        MI_ERR("alloc gestrue device failed!\n");
        goto err_request_device;
}

msa_gestrue_idev->evbit[0] = BIT_MASK(EV_SYN) | BIT_MASK(EV_KEY);
msa_gestrue_idev->name = MSA_GESTURE_DEVICE_NAME;
msa_gestrue_idev->id.bustype = BUS_I2C;

set_bit(EV_KEY, msa_gestrue_idev->evbit);
set_bit(EV_REL, msa_gestrue_idev->evbit);
set_bit(KEY_WAKEUP, msa_gestrue_idev->keybit);

err =  input_register_device(msa_gestrue_idev);
if (err) {
        MI_ERR("register msa_gestrue_idev fail");
        goto err_register_device;
}
```

这里的设备上报能力，我们只注册了需要的 KEY_WAKEUP，其余按键并未注册。如有新的按键需注册，可自行参考注册即可。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7.3.2.3 设备节点创建

创建一个 gesture_enable 的节点在 sys/class/msa 目录下。

节点处理逻辑和初始化

```
#if (LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0))
#define CONST_TYPE_PREFIX const
#else
#define CONST_TYPE_PREFIX
#endif

/*----------------------------------------------------------------------------*/
static ssize_t gesture_enable_show(CONST_TYPE_PREFIX struct class *cls,
              CONST_TYPE_PREFIX struct class_attribute *attr, char *buf)
{
        int                     ret;

        ret = sprintf(buf, "%d\n", ges_enable);
        return ret;
}
/*----------------------------------------------------------------------------*/
static ssize_t gesture_enable_store(CONST_TYPE_PREFIX struct class *cls,
              CONST_TYPE_PREFIX struct class_attribute *attr,
              const char *buf, size_t count)
{
        int                     ret;
        int                     bEnable;
        unsigned long   enable;
        MSA_HANDLE      handle = msa_handle;

        if (buf == NULL) {
                return -1;
        }

        enable = simple_strtoul(buf, NULL, 10);
        bEnable = (enable > 0) ? 1 : 0;

        if (ges_enable != bEnable) {
                ges_enable = bEnable;
                wakeup_report = ges_enable;
                input_set_int_enable(&(gsensor_info.input_type), ges_enable);
        }

        ret = msa_set_gesture_enable(handle, ges_enable);
        if (ret < 0) {
                ret = -EINVAL;
        } else {
                ret = count;
        }

        MI_ERR("msa set gesture enable %d\n", ges_enable);

        return ret;
}


static CLASS_ATTR_RW(gesture_enable);
```

```
static struct attribute *msa_class_attrs[] = {
        &class_attr_gesture_enable.attr,
        NULL
};

ATTRIBUTE_GROUPS(msa_class);

static struct class msa_class = {
        .name = "msa",
#if (LINUX_VERSION_CODE <= KERNEL_VERSION(6, 3, 0))
        .owner = THIS_MODULE,
#endif
        .class_groups = msa_class_groups,
};
```

其大致逻辑如下。

- 通过节点写入 0 和 1 来开启功能；
- 基于写入的 0 和 1 来开关中断；
- 通过 msa_set_gesture_enable 来使能 sensor 端的抬起唤醒功能。

在 probe 中创建节点

```
        err = class_register(&msa_class);
        if (err) {
                MI_ERR("create class file failed!\n");
                goto err_remove_sysfs_group;
        }
```

技巧

sysfs 节点尽量采用 class 的方式，这样可注册成/sys/class/msa/gestrue_enable 固定路径，如果注册到 input 设备下，则路径会存在变化，不利于开关使能。

7.3.2.4 中断处理

中断处理主要逻辑分 2 步：

1. 等待 resume 函数完成，再进行数据读取；
2. 抬起角度计算，并基于阈值上报唤醒事件。

7.3.2.4.1 等待 resume 完成

中断中，需等待 sensor resume 函数完成，原因是中断唤醒是最先开始触发的，而此时 i2c 或 spi 还未就绪，无法进行事件传输和读取 sensor 数据，而 i2c 需要等待 i2c resmue 完成后才可以传输数据；由于 sensor 是 i2c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

的从设备，故 i2c 的 resume 必先于 sensor 的 resume。因此只需等待 sensor 的 resume 函数完成，即可传输数据。

如下是一个采用 wait_queue_head_t 的示例。

```
if (ges_enable) {
        spin_lock_irqsave(&system_lock, flags);
        if (sleep_state == SLEEP_ON) {
                spin_unlock_irqrestore(&system_lock, flags);
                pm_wakeup_hard_event(&(msa_gestrue_idev->dev));
                pr_debug("wait system resume\n");
                t = wait_event_timeout(wait_q,
                                wait_until_wake,
                                msecs_to_jiffies(2000));
        } else {
                spin_unlock_irqrestore(&system_lock, flags);
        }
} else {
        goto out;
}
```

这里的主要逻辑如下：

1. 判断当前的 sensor 状态，是否为 SLEEP_ON，如果是则说明为休眠中唤醒，需要等待 resume 函数，如果为 OFF，则说明只是灭屏，设备未休眠；
2. pm_wakeup_hard_event：保持唤醒，避免事件还未处理完成就休眠；
3. wait_event_timeout：等待变量 wait_until_wake 为 true，或者 2000ms 后超时退出，而变量 wait_until_wake 则是在休眠唤醒中处理。

7.3.2.4.2 计算抬起角度，唤醒屏幕

在中断唤醒后，还需要计算抬起角度，以此确定是否唤醒

```
static long calculate_rotation_angle(short dx1, short dy1, short dz1, short dx2, short dy2,
    short dz2)
{
        long dot_product = (dx1 * dx2 + dy1 * dy2 + dz1 * dz2) * 10/1024;

        long magnitude_a = int_sqrt((dx1 * dx1 + dy1 * dy1 + dz1 * dz1) * 10/1024);
        long magnitude_b = int_sqrt((dx2 * dx2 + dy2 * dy2 + dz2 * dz2) * 10/1024);

        long cos_theta = dot_product * 1000 / (magnitude_a * magnitude_b);
        pr_debug("aw==== %s 1 {%d %d %d} 2 {%d %d %d}\n", __func__, dx1, dy1, dz1, dx2, dy2,
    dz2);
        pr_debug("aw==== %s dot_product=%ld, magnitude_a=%ld, magnitude_b=%ld, cos_theta=%ld\n
    ", __func__,
                        dot_product, magnitude_a, magnitude_b, cos_theta);
        return cos_theta;
}

static bool is_need_report_wakeup(void)
{
    int i = 0;
```

```
    short x = 0, y = 0, z = 0;
    short x_new = 0, y_new = 0, z_new = 0;
    long raise_angle = 0;
    MSA_HANDLE handle = msa_handle;

    pr_err("aw==== %s\n", __func__);
    for (i = 0; i < 5; i++) {
        msdelay(10);
        if (msa_read_data(handle, &x, &y, &z) != 0) {
            return false;
        }
        MI_ERR("Waiting for data stabilization, x=%d, y=%d, z=%d at iteration %d\n", x, y, z,
     i);
    }

    for (i = 0; i < 15; i++) {
        msdelay(100);
        if (msa_read_data(handle, &x_new, &y_new, &z_new) != 0) {
            continue;
        }
        raise_angle = calculate_rotation_angle(x, y, z, x_new, y_new, z_new);
        if (raise_angle <= wake_angle) {
            MI_ERR("raise angle cos %ld ,wake angle cos %ld, wake system\n", raise_angle,
   wake_angle);
            return true;
        }
    }

    return false;
}

static irqreturn_t msa_irq_handler(int irq, void *data)
{
       if (is_need_report_wakeup()) {
               input_report_key(msa_gestrue_idev, KEY_WAKEUP, 1);
               input_sync(msa_gestrue_idev);
               mdelay(100);
               input_report_key(msa_gestrue_idev, KEY_WAKEUP, 0);
               input_sync(msa_gestrue_idev);
               wakeup_report = 0;
       }
}
```

是否唤醒的主要逻辑如下：

1. 唤醒后读取第一次数据作为基准数据；
2. 在后续 1.5s 内，每隔 0.1s 读取一次数据，并基于首次读取的数据，计算角度的余弦值，当夹角余弦值大于预设的触发角度时，上报唤醒事件唤醒屏幕。

这里需要注意的点如下：

1. 由于 gsensor 首次读取的值会有异常，故在这里做了滤波，50ms 之内 5 次读取数据，第 5 次读取数据作为首次读取值；
2. 读取间隔和次数可自由配置，通常 1-1.5s 比较合适；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3. 夹角判断，采用的是首次读取值和后续读取值的夹角，如果想使用水平面的夹角，则将首次的值，替换为（0,0,1024）或（0,0,-1024）即可；
4. calculate_rotation_angle 中进行了 *10/1024，主要是将读取的值转换成类似于（0,0,9.8）范围值。不同 sensor 有不同的转换系数；
5. calculate_rotation_angle 计算的是夹角的余弦值，余弦值的取值范围是（-1，1），其中余弦值为 1 时，夹角为 0 度，余弦值为-1 时，夹角为 180 度，采用余弦值而非直接比较角度的原因是 linux 中不便于过多计算复杂的数学逻辑，会影响性能，最后计算余弦值时，先 *1000 再计算也是将（-1,1）的值放大到（-1000,1000），减少 linux 中的浮点数运算。
6. 由于余弦值越大，夹角越小，故对比角度时，采用的是 calculate_rotation_angle(x, y, z, x_new, y_new, z_new) <= wake_angle，而非 >=。

7.3.2.5 休眠唤醒处理

在手势唤醒功能打开时，需要在休眠和唤醒时，做一些特殊处理，主要关键逻辑如下：

1. 休眠时，sensor 不掉电，中断唤醒需打开，同时还需要打开一些需要的驱动变量；
2. 唤醒时，设置对应的 flag，并通知中断可以继续处理。

主要代码如下：

```
/*-----------------------------------------------------------------------*/
static int msa_suspend(struct device *dev)
{
        int result  =  0;
        unsigned long flags;
        MSA_HANDLE      handle = msa_handle;

        MI_FUN;

        pr_err("aw==== %s\n", __func__);

        if (ges_enable) {
                pr_err("suspend to gestrue wakeup");
                //suspend
                spin_lock_irqsave(&system_lock, flags);
                wait_until_wake = false;
                sleep_state = SLEEP_ON;
                spin_unlock_irqrestore(&system_lock, flags);
        } else {
                result = msa_set_enable(handle, 0);
                if (result) {
                        MI_ERR("%s:set disable fail!!\n",   __func__);
                        return result;
                }

                input_set_power_enable(&(gsensor_info.input_type),   0);
        }
        return result;
}
/*-----------------------------------------------------------------------*/
static int msa_resume(struct device *dev)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
{
        int result = 0;
        unsigned long flags;
        MSA_HANDLE      handle = msa_handle;

        MI_FUN;
        if (ges_enable) {
                pr_err("resume from gestrue wakeup");
                //resume
                spin_lock_irqsave(&system_lock, flags);
                wait_until_wake = true;
                sleep_state = SLEEP_OFF;
                spin_unlock_irqrestore(&system_lock, flags);
                wake_up(&wait_q);
        } else {
                input_set_power_enable(&(gsensor_info.input_type),   1);
                result  =  msa_chip_resume(handle);

                if (result) {
                        MI_ERR("%s:chip resume fail!!\n", __func__);
                        return result;
                }

                result  =  msa_set_enable(handle, 1);
                if (result) {
                        MI_ERR("%s:set enable fail!!\n", __func__);
                        return result;
                }

        }
        return result;
}
```

7.3.3 效果调优

接下来主要以 msa321 sensor 为例，描述对应的驱动效果调优，变量设置等。

⚠ 注意

效果调优需结合实际的 sensor 进行。

7.3.3.1 中断触发阈值调整

中断触发阈值，即 sensor 对于设备拿起动作的敏感度，敏感度越高，设备越容易唤醒，但同时也会出现误唤醒，导致功耗增大；敏感度低，则触发不够灵敏，功能容易有 BUG，故需要调整合适的触发阈值。

在 msa321sensor 中，阈值调整寄存器为 0x28，其值越低，敏感度越高。默认为 0xA，当前采取的触发阈值为 0x2，如果是 0x1 则非常容易唤醒。

调整方法在 msa_core.c 的 msa_set_gesture_enable 方法中。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
res = msa_register_write(handle, 0x28, 0x02);
```

修改后面的 0x2 数字，即可得到不同的触发阈值。

7.3.3.2 唤醒角度配置

唤醒角度，主要是检测抬起夹角的阈值，这里采用的余弦值 *1000 的值。余弦值可通过对应的角度并使用计算器计算。

余弦值的取值范围是（-1，1），其中余弦值为 1 时，夹角为 0 度，余弦值为-1 时，夹角为 180 度，采用余弦值而非直接比较角度的原因是 linux 中不便于过多计算复杂的数学逻辑，会影响性能，最后计算余弦值值时，先 *1000 再计算也是将（-1,1）的值放大到（-1000,1000），减少 linux 中的浮点数运算。

以 30 度为例，其余弦值为 cos(30°)=0.866，乘以 1000 为 866。

在 msa321sensor 中，唤醒角度配置可通过 msa_cust.c 中的变量 wake_angle 配置，当前配置为 850，抬起角度大概在 32 度。

```
long wake_angle = 850;
```

7.3.3.3 检测时长配置

当前 msa321 中，检测次数是 15 次，采样间隔为 100ms，故在检测到中断后，整体监测角度变化时间为 1.5s。如果在 1.5s 内抬起角度大于触发阈值，则会唤醒，反之则继续休眠。

采样次数和采样间隔可在 msa_cust.c 中的 is_need_report_wakeup 函数中配置。

```
static bool is_need_report_wakeup(void)
{
    for (i = 0; i < 15; i++) {
        msdelay(100);
    }
}
```

其中，for 循环中的 15 表示采样次数，msdelay 为采样间隔，二者相乘为总的采样时间。

7.4 Android 抬起唤醒功能支持

Android 抬起唤醒支持的前提条件是，驱动抬起唤醒功能正常，可通过节点写入的方式控制开关。

7.4.1 方案配置开关

Android 主要有 2 个配置：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1. 菜单显示：配置是否显示抬起唤醒开关；如未配置，则默认不显示菜单；
2. 功能默认值：如配置成 true，则默认打开抬起唤醒。否则默认关闭

7.4.1.1 菜单显示

菜单是否显示，通过`config_enable_raise_to_wake`来进行开关配置，其配置方法如下。

1. 在对应的 Android 方案配置下，找到如下文件。

以 A537 为例，方案配置路径。

```
device/softwinner/earth
```

则对应的开发 feature 文件路径。

```
common/overlay/overlay_rro/vendor/aw/public/package/rro/AwPadFeatureConfig/res/values/config.
    xml
```

说明

如果没有该文件，可自行创建对应文件。

2. 增加配置，显示菜单。

```
<resources xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:xliff="urn:oasis:names:tc:xliff:document:1.2">
    <bool name="config_enable_raise_to_wake">true</bool>
</resources>
```

如上，将`config_enable_raise_to_wake`配置为 true，即可在 setting 应用中，显示抬起唤醒的菜单配置。

7.4.1.2 功能默认值

方案中，还可通过配置`def_raise_to_wake_enabled`来默认打开抬起唤醒；功能默认关闭。

如果需要默认打开抬起唤醒功能，可参考下列配置步骤。

1. 在对应的 Android 方案配置下，找到如下文件

以 A537 为例，方案配置路径。

```
device/softwinner/earth
```

则对应的开发 feature 文件路径。

```
common/overlay/overlay/frameworks/base/packages/SettingsProvider/res/values/defaults.xml
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

如果没有该文件，可自行创建对应文件。

2. 默认使能抬起唤醒。

```
    <bool name="def_raise_to_wake_enabled">true</bool>
```

如上，将`def_raise_to_wake_enabled`配置为 true，即可在首次开机后，默认启用抬起唤醒功能。

7.4.2 Powerhal 配置文件节点

在 Powerhal 中，配置抬起唤醒的配置，连接 Android 和 Linux 的通路。

以 A537 平台为例，增加抬起唤醒的配置文件。

配置文件目录路径

```
hardware/aw/power/aidl/configs/sun65iw1p1
```

在`aw_power_scene_config.xml`增加抬起唤醒场景

```
<scene name="raise_to_wake">
    <set resource="msa_gesture_enable" value="enable"/>
</scene>
```

注意

抬起唤醒的 scene 名字必须为 raise_to_wake，否则会匹配失败。

在`aw_power_resource.xml`中，增加 msa_gesture_enable 资源

```
<resource name="msa_gesture_enable" type="sysfs" default="disable">
    <config name="enable"  priority="1">
        <write path="/sys/class/msa/gesture_enable" value="1"/>
    </config>
    <config name="disable"  priority="2">
        <write path="/sys/class/msa/gesture_enable" value="0"/>
    </config>
</resource>
```

关于 resource 的资源说明如下：

1. 抬起唤醒控制路径为`/sys/class/msa/gesture_enable`；
2. 其中 1 表示使能抬起唤醒，0 表示关闭；
3. 不同 sensor 的使能节点路径可能不同，可修改为其他配置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

部分 SDK 中，还需要在`hardware/aw/power/aidl/configs/`的 aw_power_resource_id_define.txt 和 aw_power_scene_id_define.txt 增加对应的 id。

7.4.3 使用说明

在方案配置打开菜单显示后，在 Settings–>Pad Features–>ConVenient Wake Up 菜单可看到 Raise to Wake 选项。

广州星翼电子科技有限公司zdyzlinux

图 7-2: raise_to_wake

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8 调试方法

8.1 dumpsys sensorservice

通过 dumpsys sensorservice 可获取相关的 sensor 信息。

主要的信息如下。

1. 当前的sensor 信息

```
a523-pro:/ # dumpsys sensorservice
Captured at: 13:23:17.866
Sensor Device:
Total 4 h/w sensors, 4 running 0 disabled clients:
Sensor List:
0x00000001) Mi3da 3-axis Accelerometer | miramems Semiconductor Inc. | ver: 1 | type: android
    .sensor.accelerometer(1) | perm: n/a | flags: 0x00000000
        continuous | minRate=1.00Hz | maxRate=100.00Hz | no batching | non-wakeUp |
0x00000002) Mi3da 3-axis Accelerometer uncalibrated | miramems Semiconductor Inc. | ver: 1 |
    type: android.sensor.accelerometer_uncalibrated(35) | perm: n/a | flags: 0x00000000
        continuous | minRate=1.00Hz | maxRate=100.00Hz | no batching | non-wakeUp |
0x00000003) Memsic MMC5603NJ 3-axis Magnetic field sensor | Memsic MMC5603NJ | ver: 1 | type:
     android.sensor.magnetic_field(2) | perm: n/a | flags: 0x00000000
        continuous | minRate=1.00Hz | maxRate=100.00Hz | no batching | non-wakeUp |
0x00000004) Memsic MMC5603NJ 3-axis Magnetic field sensor uncalibrated | Memsic MMC5603NJ |
    ver: 1 | type: android.sensor.magnetic_field_uncalibrated(14) | perm: n/a | flags: 0
    x00000000
        continuous | minRate=1.00Hz | maxRate=100.00Hz | no batching | non-wakeUp |
0x5f67656f) GeoMag Rotation Vector Sensor | AOSP            | ver: 3 | type: android.sensor.
    geomagnetic_rotation_vector(20) | perm: n/a | flags: 0x00000000
        continuous | maxDelay=0us | maxRate=100.00Hz | no batching | non-wakeUp |
Fusion States:
9-axis fusion disabled (0 clients), gyro-rate= 200.00Hz, q=< 0, 0, 0, 0 > (0), b=< 0, 0, 0 >
game fusion(no mag) disabled (0 clients), gyro-rate= 200.00Hz, q=< 0, 0, 0, 0 > (0), b=< 0,
    0, 0 >
geomag fusion (no gyro) disabled (0 clients), gyro-rate= 200.00Hz, q=< 0, 0, 0, 0 > (0), b=<
    0, 0, 0 >
```

2. 最近的 50 个 sensor 事件。

```
Recent Sensor events:
Mi3da 3-axis Accelerometer: last 50 events
         1 (ts=449.681903088, wall=11:20:08.027) -0.13, 6.75, 7.10,
         2 (ts=449.746894046, wall=11:20:08.092) -0.13, 6.76, 7.07,
         3 (ts=449.811990255, wall=11:20:08.157) -0.13, 6.80, 7.05,
         4 (ts=449.877012046, wall=11:20:08.222) -0.13, 6.78, 7.05,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
         5 (ts=449.877068005, wall=11:20:08.222) -0.15, 6.78, 7.08,
         6 (ts=449.942762088, wall=11:20:08.288) -0.16, 6.79, 7.07,
         7 (ts=450.007137171, wall=11:20:08.352) -0.13, 6.76, 7.10,
         8 (ts=450.072196380, wall=11:20:08.418) -0.12, 6.78, 7.08,
         9 (ts=450.137091588, wall=11:20:08.482) -0.12, 6.81, 7.08,
        10 (ts=450.137176130, wall=11:20:08.482) -0.13, 6.81, 7.02,
```

3. 当前活跃的 sensor。

```
Active sensors:
Socket Buffer size = 39 events
WakeLock Status: not held
Mode : NORMAL
Sensor Privacy: disabled
0 active connections
0 direct connections
```

4. 监听 sensor 的历史进程信息。

```
Previous Registrations:
11:20:10 - 0x00000001 pid=  619 uid= 1000 package=com.android.server.wm.
    WindowOrientationListener
11:20:00 + 0x00000001 pid=  619 uid= 1000 package=com.android.server.wm.
    WindowOrientationListener samplingPeriod=66667us batchingPeriod=100000us
11:19:59 - 0x00000001 pid=  619 uid= 1000 package=com.android.server.wm.
    WindowOrientationListener
11:19:29 + 0x00000001 pid=  619 uid= 1000 package=com.android.server.wm.
    WindowOrientationListener samplingPeriod=66667us batchingPeriod=100000us
```

8.2 remonut 后推库

修改代码后，可通过推库的方式替换对应文件，重启观察效果。

```
adb root
adb remount
```

可修改的有。

1. 编译的 bin 文件：vendor/bin/hw/android.hardware.sensors-service.aw。
2. 启动 sensorhal 的 rc 文件：vendor/etc/init/sensors-default-aw.rc。
3. gsensor 方向配置文件：vendor/etc/gsensor.cfg。
4. sensor 模块加载文件：vendor/etc/sensor_modules.cfg。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9 常见问题

9.1 dumpsys sensorservice 能看到设备信息，app 却无法正确获得 sensor

排查是否是 sensorinfo 信息存在错误，如 delay 错误的设置了 0，sensor 类型 flag 设置错误。

1. 通常 delay 不应该设置为 0。
2. flag 针对如 gsensor 等连续上报数据的 sensor，应设置为 SENSOR_FLAG_CONTINUOUS_MODE，如距离感应等，应设置为 SENSOR_FLAG_ON_CHANGE_MODE。

9.2 节点无法写入，提示没有权限

sensor 相关节点，主要有 enable 和 delay，该节点是驱动创建的，当前可通过 uevent 机制修改相关节点权限。

以 mir3da 驱动，节点为 enable 和 delay，适配步骤参考如下。

1. 在/sys/class/input 目录下通过`ls -l`获取正确的节点路径并查看对应的节点权限信息，其中 input3 为 mir3da 驱动。

```
a523-pro:/sys/class/input # ls -l
total 0
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 event0 -> ../../devices/platform/soc@3000000/7081400.s_twi0/i2c-6/6-0034/axp2101-pek.0/input/input0/event0
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 event1 -> ../../devices/platform/soc@3000000/soc@3000000:codec_mach/sound/card0/input1/event1
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 event2 -> ../../devices/platform/soc@3000000/soc@3000000:daudio0_mach/sound/card1/input2/event2
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 event3 -> ../../devices/virtual/input/input3/event3
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 event4 -> ../../devices/virtual/input/input4/event4
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 event5 -> ../../devices/platform/soc@3000000/2009800.lradc/input/input5/event5
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 event6 -> ../../devices/virtual/input/input6/event6
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 event7 -> ../../devices/virtual/input/input7/event7
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 input0 -> ../../devices/platform/soc@3000000/7081400.s_twi0/i2c-6/6-0034/axp2101-pek.0/input/input0
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 input1 -> ../../devices/platform/soc@3000000/soc@3000000:codec_mach/sound/card0/input1
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 input2 -> ../../devices/platform/soc@3000000/soc@3000000:daudio0_mach/sound/card1/input2
lrwxrwxrwx 1 root root 0 2023-03-16 11:12 input3 -> ../../devices/virtual/input/input3
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 input4 -> ../../devices/virtual/input/input4
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 input5 -> ../../devices/platform/soc@3000000/2009800.lradc/input/input5
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 input6 -> ../../devices/virtual/input/input6
lrwxrwxrwx 1 root root 0 2023-03-16 14:14 input7 -> ../../devices/virtual/input/input7
a523-pro:/sys/class/input #
```

图 9-1: 节点路径

如图所示，可获取到 mir3da 驱动正确的节点路径是/sys/devices/virtual/input/input3。

2. 在方案目录的 uevent.rc 中，添加对应的节点信息，以 Android 13 为例，路径如下。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
device/softwinner/saturn/common/system/ueventd.sun55iw3p1.rc
```

对应添加的 uevent 信息如下。

```
/dev/teepriv0              0776    media       mediadrm
/dev/vipcore               0666    system      system
/dev/galcore               0666    system      system
/dev/input/*                0660    system      input
/dev/sunxi_drm_heap        0777    system      graphics
/dev/graphics/fb0          0666    system      graphics
/dev/deinterlace           0666    media       media

# sysfs properties
/sys/devices/virtual/input/input*   delay          0660  system   system
/sys/devices/virtual/input/input*   enable         0660  system   system
/sys/devices/virtual/input/input*   ps_poll_delay  0660  system   system
/sys/devices/virtual/input/input*   ls_poll_delay  0660  system   system
```

图 9-2: uevent

参考为。

```
/sys/devices/virtual/input/input*   delay          0660  system   system
/sys/devices/virtual/input/input*   enable         0660  system   system
```

其表示为，针对/sys/devices/virtual/input/input* 下的所有 delay 和 enable 节点，修改为 system 用户切权限为 660。

3. 在驱动创建节点位置，上报 uevent 告知节点变动。

```
result = sysfs_create_group(&idev->dev.kobj, &mir3da_attr_group);      //注册节点信息
if (result) {
        MI_ERR("create device file failed!\n");
        result = -EINVAL;
        goto err_unregister_polled_device;
 }

kobject_uevent(&idev->dev.kobj, KOBJ_CHANGE);        //固定用法，上报节点变动信息，注意传入参数
```

对于其他名称不同节点，如 ps_poll_delay，方法一致。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。